

Inventory Management & Sales Analysis

SQL & Python Business Analytics Project

Objective:

Analyze sales and inventory data to identify inventory inefficiencies, stock shortage drivers, and demand patterns, and translate the findings into actionable business recommendations.

Tools:

Excel | SQL Server | Python | Pandas | Matplotlib | Statistical Analysis

1. Setup

Install the required packages to connect directly to a SQL Server database from within Jupyter using `ipython-sql`.

```
In [1]: %pip install ipython-sql sqlalchemy pyodbc

Requirement already satisfied: ipython-sql in .\anaconda3\Lib\site-packages (0.5.0)
Requirement already satisfied: sqlalchemy in .\anaconda3\Lib\site-packages (2.0.51)
Requirement already satisfied: pyodbc in .\anaconda3\Lib\site-packages (5.3.0)
Requirement already satisfied: prettytable in .\anaconda3\Lib\site-packages (from ipython-sql) (3.11.0)
Requirement already satisfied: ipython in .\anaconda3\Lib\site-packages (from ipython-sql) (9.15.0)
Requirement already satisfied: sqlparse in .\anaconda3\Lib\site-packages (from ipython-sql) (0.6.0)
Requirement already satisfied: six in .\anaconda3\Lib\site-packages (from ipython-sql) (1.17.0)
Requirement already satisfied: ipython-genutils in .\anaconda3\Lib\site-packages (from ipython-sql) (0.2.0)
Requirement already satisfied: greenlet>=1 in .\anaconda3\Lib\site-packages (from sqlalchemy) (3.3.2)
Requirement already satisfied: typing-extensions>=4.6.0 in .\anaconda3\Lib\site-packages (from sqlalchemy) (4.16.0)
Requirement already satisfied: colorama>=0.4.4 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (0.4.6)
Requirement already satisfied: decorator>=5.1.0 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (5.3.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (1.1.1)
Requirement already satisfied: jedi>=0.18.2 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (0.19.2)
Requirement already satisfied: matplotlib-inline>=0.1.6 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (0.2.2)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (3.0.52)
Requirement already satisfied: psutil>=7 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (7.2.2)
Requirement already satisfied: pygments>=2.14.0 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (2.20.0)
Requirement already satisfied: stack_data>=0.6.0 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in .\anaconda3\Lib\site-packages (from ipython->ipython-sql) (5.15.0)
Requirement already satisfied: wcwidth in .\anaconda3\Lib\site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython->ipython-sql) (0.8.2)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in .\anaconda3\Lib\site-packages (from jedi>=0.18.2->ipython->ipython-sql) (0.8.7)
Requirement already satisfied: executing>=1.2.0 in .\anaconda3\Lib\site-packages (from stack_data>=0.6.0->ipython->ipython-sql) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in .\anaconda3\Lib\site-packages (from stack_data>=0.6.0->ipython->ipython-sql) (3.0.1)
Requirement already satisfied: pure_eval in .\anaconda3\Lib\site-packages (from stack_data>=0.6.0->ipython->ipython-sql) (0.2.3)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: %load_ext sql
```

```
In [3]: %sql mssql+pyodbc://localhost/Inventory_Data?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
```

2. Inventory Performance Analysis

This section evaluates inventory efficiency across product categories and identifies areas where inventory may be underutilized or exposed to operational risk.

2.1 What is the inventory turnover rate for each category?

Question: Which categories move their inventory efficiently, and which ones accumulate stock without enough movement?

```
In [4]: %%sql result <<

SELECT
    Category,
    SUM([Units_Sold]) AS Total_Units_Sold,
    SUM([Inventory_Level]) AS Total_Inventory,
    CAST(
        SUM([Units_Sold]) * 1.0 /
        NULLIF(SUM([Inventory_Level]), 0)
        AS DECIMAL(10,2)
```

```

    ) AS Inventory_Turnover
FROM dbo.sales_data
GROUP BY Category
ORDER BY Inventory_Turnover DESC;

```

* mssql+pyodbc://localhost/Inventory_Data?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
 Done.
 Returning data to local variable result

```

In [5]: import matplotlib.pyplot as plt

plot_data = result.DataFrame()

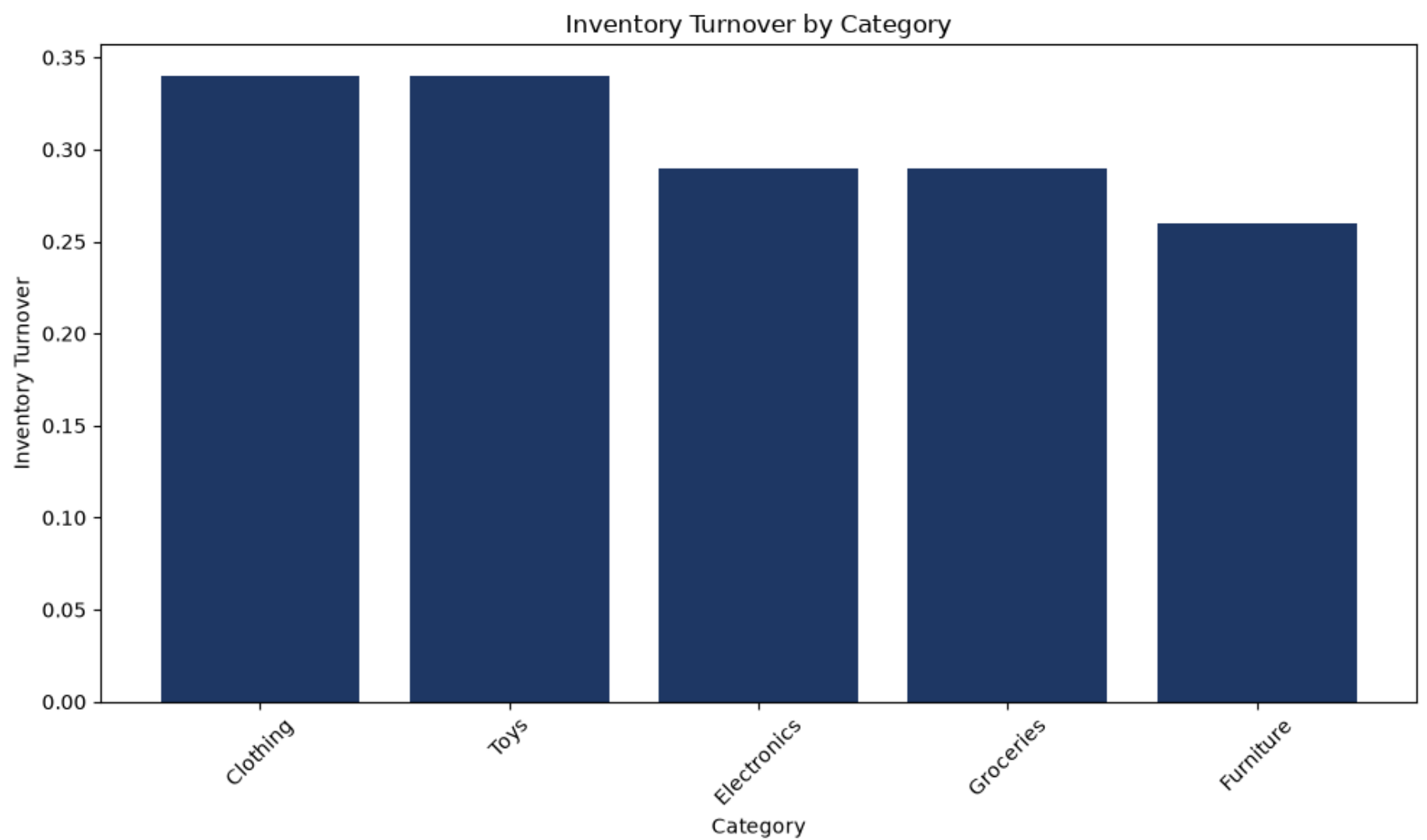
plt.figure(figsize=(10, 6))

plt.bar(
    plot_data['Category'],
    plot_data['Inventory_Turnover'],
    color='#1F3864'
)

plt.title('Inventory Turnover by Category')
plt.xlabel('Category')
plt.ylabel('Inventory Turnover')
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()

```



✦ Insight

Groceries carries by far the largest inventory volume (10.9M units) but with a relatively low turnover rate (0.29) — meaning a large amount of capital sits idle without being cleared fast enough. This partly explains why the category ranks among the worst for stock shortages despite its sheer size.

2.2 Which store is driving the shortage problem within Groceries?

Question: Is the problem concentrated in a single store, or spread across all stores?

```

In [6]: %%sql result <<

SELECT
    [Store_ID],
    COUNT(*) AS Stock_Shortage_Count
FROM dbo.sales_data
WHERE Category = 'Groceries'
    AND [Inventory_Status] = 'Stock Shortage'
GROUP BY [Store_ID]
ORDER BY Stock_Shortage_Count DESC;

```

```
* mssql+pyodbc://localhost/Inventory_Data?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.
Returning data to local variable result
```

```
In [7]: import matplotlib.pyplot as plt

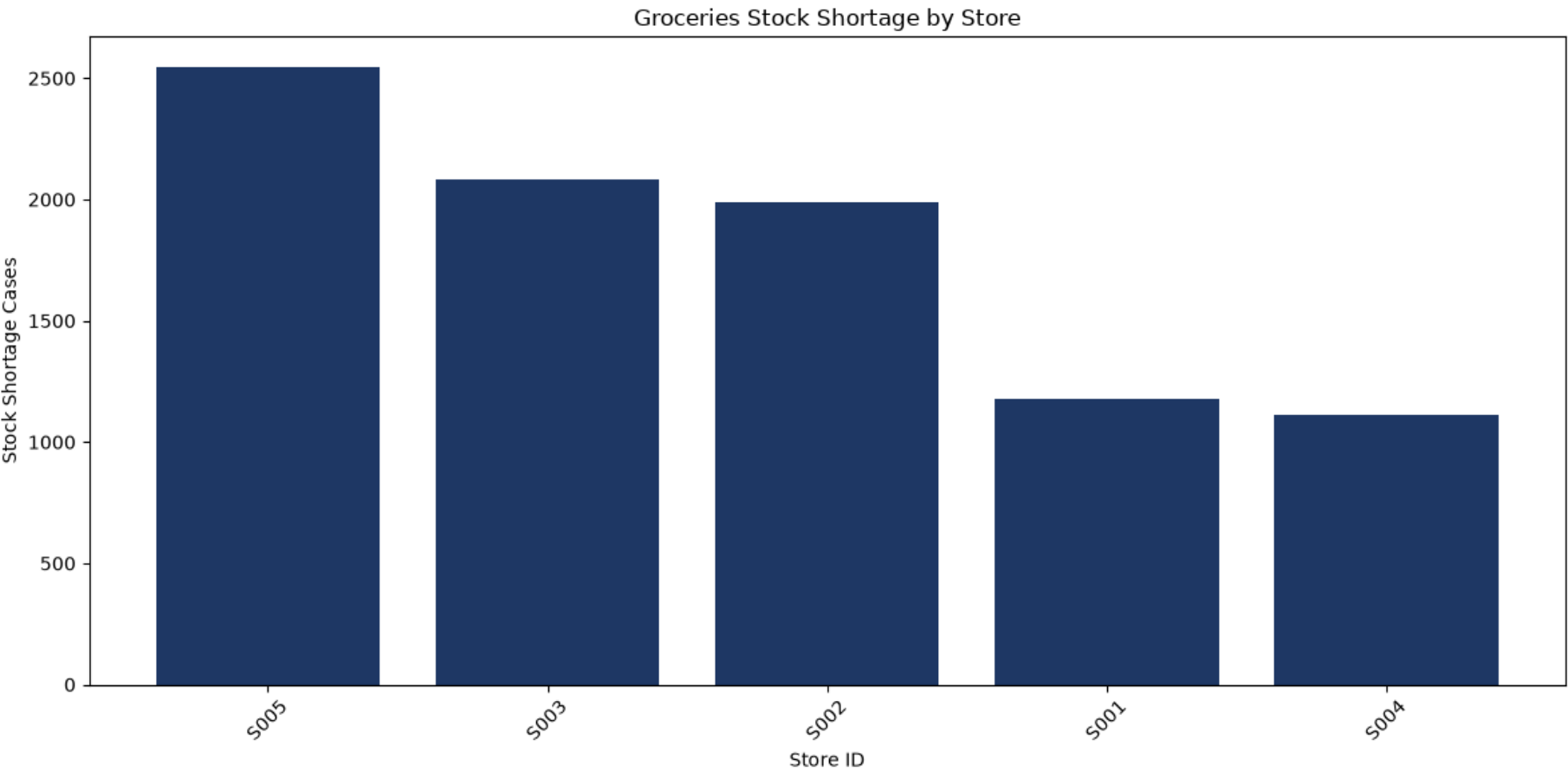
plot_data = result.DataFrame()

plt.figure(figsize=(12, 6))

plt.bar(
    plot_data['Store_ID'].astype(str),
    plot_data['Stock_Shortage_Count'],
    color='#1F3864'
)

plt.title('Groceries Stock Shortage by Store')
plt.xlabel('Store ID')
plt.ylabel('Stock Shortage Cases')
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```



✦ **Insight**

The shortage problem is distributed across all five stores without exception, though stores S005 and S003 record the highest number of shortage incidents (2,544 and 2,081 respectively). This wide distribution indicates a systemic issue in how the category is managed, rather than a management failure specific to one store.

2.3 Which products are most affected within each category?

Question: Using a within-category ranking (`RANK() PARTITION BY`), what are the actual priorities for intervention? Rather than relying on a company-wide ranking, the window function `RANK()` with `PARTITION BY` was used to identify the most affected products within each category individually — revealing that the true priority order differs from the overall company ranking.

```
In [8]: %%sql result <<

SELECT
    Category,
    Product_ID,
    Stock_Shortage_Count,
    RANK() OVER (
        PARTITION BY Category
        ORDER BY Stock_Shortage_Count DESC
    ) AS Rank_Within_Category
FROM (
    SELECT
        Category,
        Product_ID,
        COUNT(*) AS Stock_Shortage_Count
    FROM dbo.sales_data
    WHERE Inventory_Status = 'Stock Shortage'
    GROUP BY Category, Product_ID
) AS grouped_data;
```

```
* mssql+pyodbc://localhost/Inventory_Data?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.
Returning data to local variable result
```

```
In [9]: import matplotlib.pyplot as plt

plot_data = result.DataFrame()

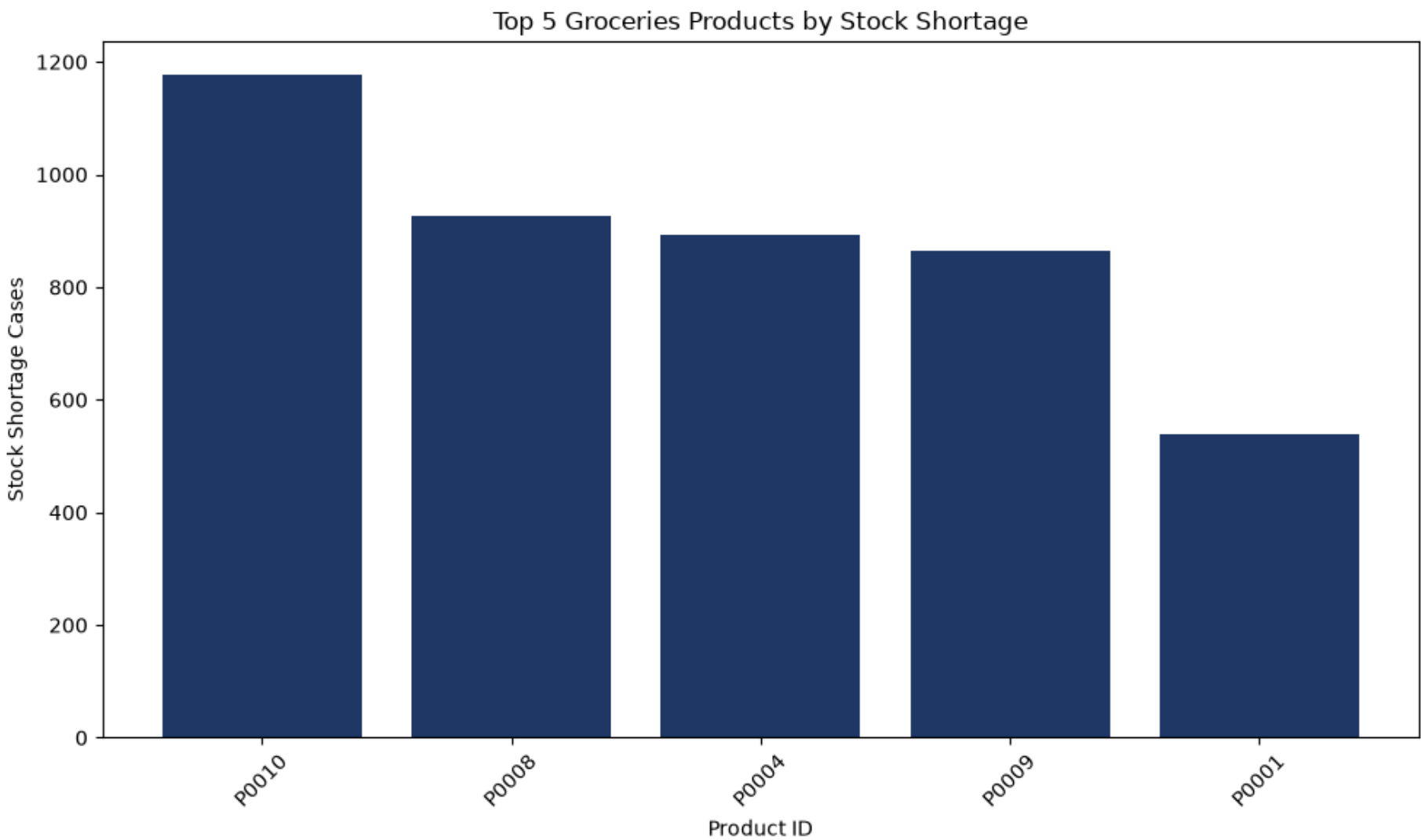
groceries_top5 = (
    plot_data[
        (plot_data['Category'] == 'Groceries') &
        (plot_data['Rank_Within_Category'] <= 5)
    ]
    .sort_values('Stock_Shortage_Count', ascending=False)
)

plt.figure(figsize=(10, 6))

plt.bar(
    groceries_top5['Product_ID'].astype(str),
    groceries_top5['Stock_Shortage_Count'],
    color='#1F3864'
)

plt.title('Top 5 Groceries Products by Stock Shortage')
plt.xlabel('Product ID')
plt.ylabel('Stock Shortage Cases')
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```



✦ Insight

Product P0010 is the most affected item specifically within the Groceries category (1,177 shortage incidents), followed by P0008, P0004, P0009, and P0001. This precise ranking differs from the company-wide ranking, confirming the importance of analyzing each category on its own before making reorder decisions.

3. Exploratory Correlation & Statistical Testing — Python

After scoping the problem via SQL, the analysis moved to Python to statistically test the possible hypotheses: is the root cause on the demand side, or on the supply side (ordering)?

Setting up the environment and loading the data

```
In [10]: import pandas as pd
import numpy as np
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [11]: df=pd.read_csv(r"D:\sales_data.csv")
```

```
In [12]: df.head()
```

Out[12]:

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Discount	Weather Condition	Promotion	Competitor Pricing	Seasonality
0	2022-01-01	S001	P0001	Electronics	North	195	102	252	72.72	5	Snowy	0	85.73	Winter
1	2022-01-01	S001	P0002	Clothing	North	117	117	249	80.16	15	Snowy	1	92.02	Winter
2	2022-01-01	S001	P0003	Clothing	North	247	114	612	62.94	10	Snowy	1	60.08	Winter
3	2022-01-01	S001	P0004	Electronics	North	139	45	102	87.63	10	Snowy	0	85.19	Winter
4	2022-01-01	S001	P0005	Groceries	North	152	65	271	54.41	0	Snowy	0	51.63	Winter

3.1 Do time-based or marketing factors explain the sales fluctuation?

Question: Do time, day of week, or promotions correlate with sales volume strongly enough to rely on? A correlation matrix was examined across all numeric variables to test whether any time-based or marketing factor shows a meaningful linear relationship with sales volume.

```
In [13]: df['Date'] = pd.to_datetime(df['Date'])
```

```
In [14]: num_data = df.select_dtypes(include=['number', 'datetime'])
```

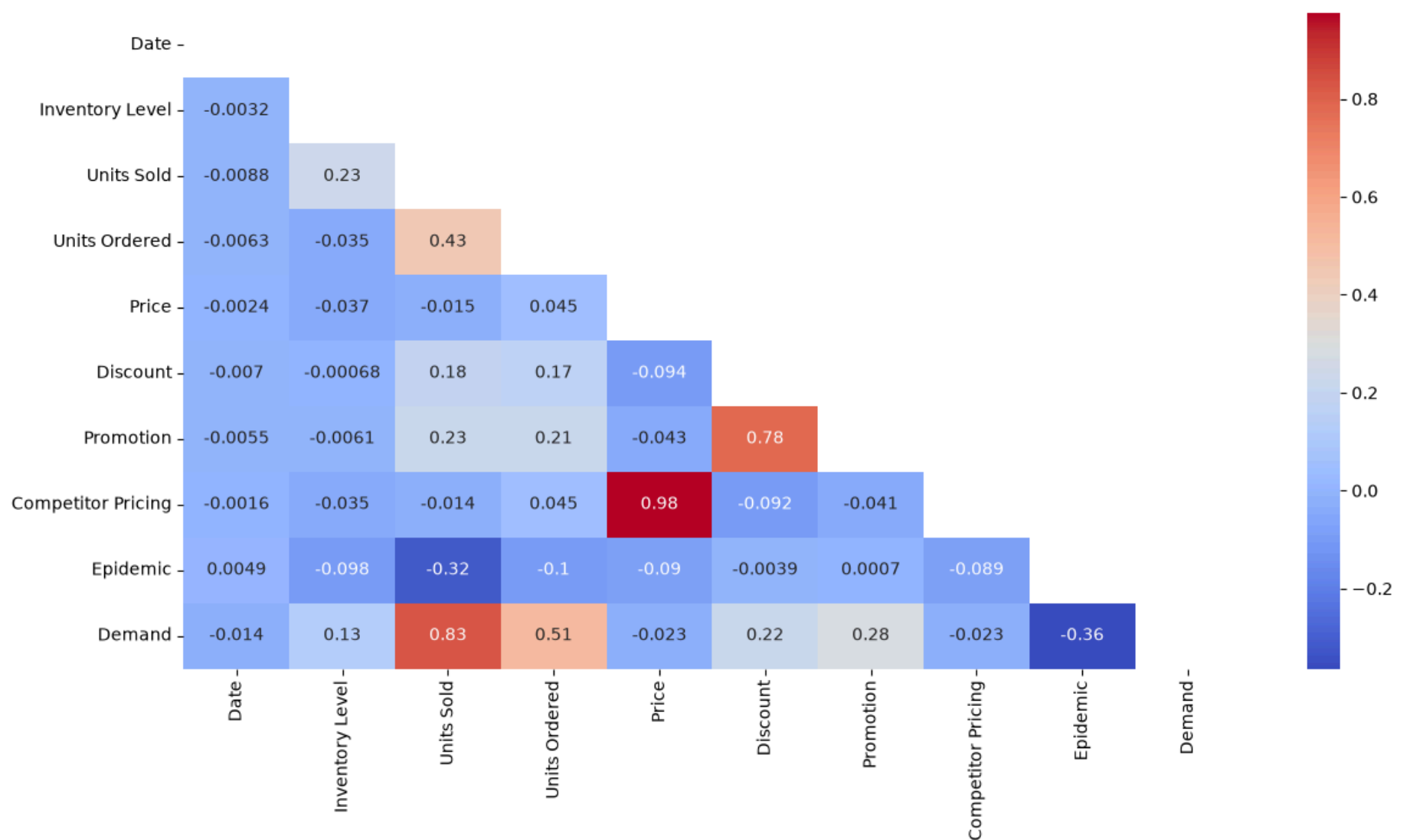
```
In [15]: corr=num_data.corr()
corr
```

Out[15]:

	Date	Inventory Level	Units Sold	Units Ordered	Price	Discount	Promotion	Competitor Pricing	Epidemic	Demand
Date	1.000000	-0.003153	-0.008827	-0.006296	-0.002404	-0.006971	-0.005529	-0.001573	0.004888	-0.014310
Inventory Level	-0.003153	1.000000	0.234590	-0.034698	-0.036673	-0.000678	-0.006082	-0.034754	-0.097550	0.126618
Units Sold	-0.008827	0.234590	1.000000	0.433943	-0.014506	0.183523	0.227062	-0.013989	-0.317855	0.833421
Units Ordered	-0.006296	-0.034698	0.433943	1.000000	0.044980	0.172972	0.214743	0.045235	-0.100546	0.511963
Price	-0.002404	-0.036673	-0.014506	0.044980	1.000000	-0.094136	-0.043190	0.976648	-0.089658	-0.023461
Discount	-0.006971	-0.000678	0.183523	0.172972	-0.094136	1.000000	0.784039	-0.092066	-0.003885	0.224723
Promotion	-0.005529	-0.006082	0.227062	0.214743	-0.043190	0.784039	1.000000	-0.041385	0.000700	0.282537
Competitor Pricing	-0.001573	-0.034754	-0.013989	0.045235	0.976648	-0.092066	-0.041385	1.000000	-0.089466	-0.023036
Epidemic	0.004888	-0.097550	-0.317855	-0.100546	-0.089658	-0.003885	0.000700	-0.089466	1.000000	-0.363661
Demand	-0.014310	0.126618	0.833421	0.511963	-0.023461	0.224723	0.282537	-0.023036	-0.363661	1.000000

```
In [16]: plt.figure(figsize=(14,7))
sns.heatmap(corr, annot= True ,cmap='coolwarm',mask=np.triu(corr))
```

```
Out[16]: <Axes: >
```



✦ Insight

The correlation analysis confirmed the absence of any meaningful relationship between time or the available marketing factors and sales volume. This rules out the hypothesis that the inventory shortage stems from a shift in demand patterns, and redirects the investigation toward the supply side (ordering) rather than the demand side.

3.2 Is the ordered quantity actually sufficient to cover consumption?

Question: What is the ratio of Units Ordered to Units Sold within each category? The order-to-sales ratio (Buffer Ratio) was calculated for each category, measuring whether a real safety margin exists between the quantity ordered and the quantity actually consumed.

```
In [17]: groceries = df[df['Category'] == 'Groceries'].copy()

comparison_all = groceries.groupby('Store ID')[['Units Sold', 'Units Ordered']].mean()
comparison_all['Buffer_Ratio'] = comparison_all['Units Ordered'] / comparison_all['Units Sold']
print(comparison_all)
```

Store ID	Units Sold	Units Ordered	Buffer_Ratio
S001	103.980263	104.520614	1.005197
S002	101.982401	102.179441	1.001932
S003	101.729678	102.104240	1.003682
S004	104.684211	105.258835	1.005489
S005	102.681711	102.895789	1.002085

```
In [18]: all_categories = df.groupby(['Category', 'Store ID'])[['Units Sold', 'Units Ordered']].mean()
all_categories['Buffer_Ratio'] = all_categories['Units Ordered'] / all_categories['Units Sold']

print(all_categories.round(4))
```

		Units Sold	Units Ordered	Buffer_Ratio
Category	Store ID			
Clothing	S001	104.3588	104.7539	1.0038
	S002	90.3105	90.6842	1.0041
	S003	94.6329	95.0178	1.0041
	S004	93.3747	93.4479	1.0008
	S005	93.5382	93.4171	0.9987
Electronics	S001	78.0618	78.0706	1.0001
	S002	82.9338	83.6794	1.0090
	S003	91.5627	91.5706	1.0001
	S004	74.6000	74.5816	0.9998
	S005	82.1092	82.7743	1.0081
Furniture	S001	65.0724	65.1757	1.0016
	S002	64.2421	64.3974	1.0024
	S003	64.6377	65.0232	1.0060
	S004	63.8630	63.9718	1.0017
	S005	64.4232	64.5360	1.0017
Groceries	S001	103.9803	104.5206	1.0052
	S002	101.9824	102.1794	1.0019
	S003	101.7297	102.1042	1.0037
	S004	104.6842	105.2588	1.0055
	S005	102.6817	102.8958	1.0021
Toys	S001	75.7385	75.7273	0.9999
	S002	81.1316	81.4417	1.0038
	S003	78.8610	78.9912	1.0017
	S005	78.8326	78.8855	1.0007

```
In [19]: category_summary = df.groupby('Category')[['Units Sold', 'Units Ordered']].mean()
category_summary['Buffer_Ratio'] = category_summary['Units Ordered'] / category_summary['Units Sold']
print(category_summary.round(4))
```

	Units Sold	Units Ordered	Buffer_Ratio
Category			
Clothing	94.6442	94.8984	1.0027
Electronics	83.0411	83.3410	1.0036
Furniture	64.3753	64.5322	1.0024
Groceries	102.8729	103.2317	1.0035
Toys	78.4473	78.5536	1.0014

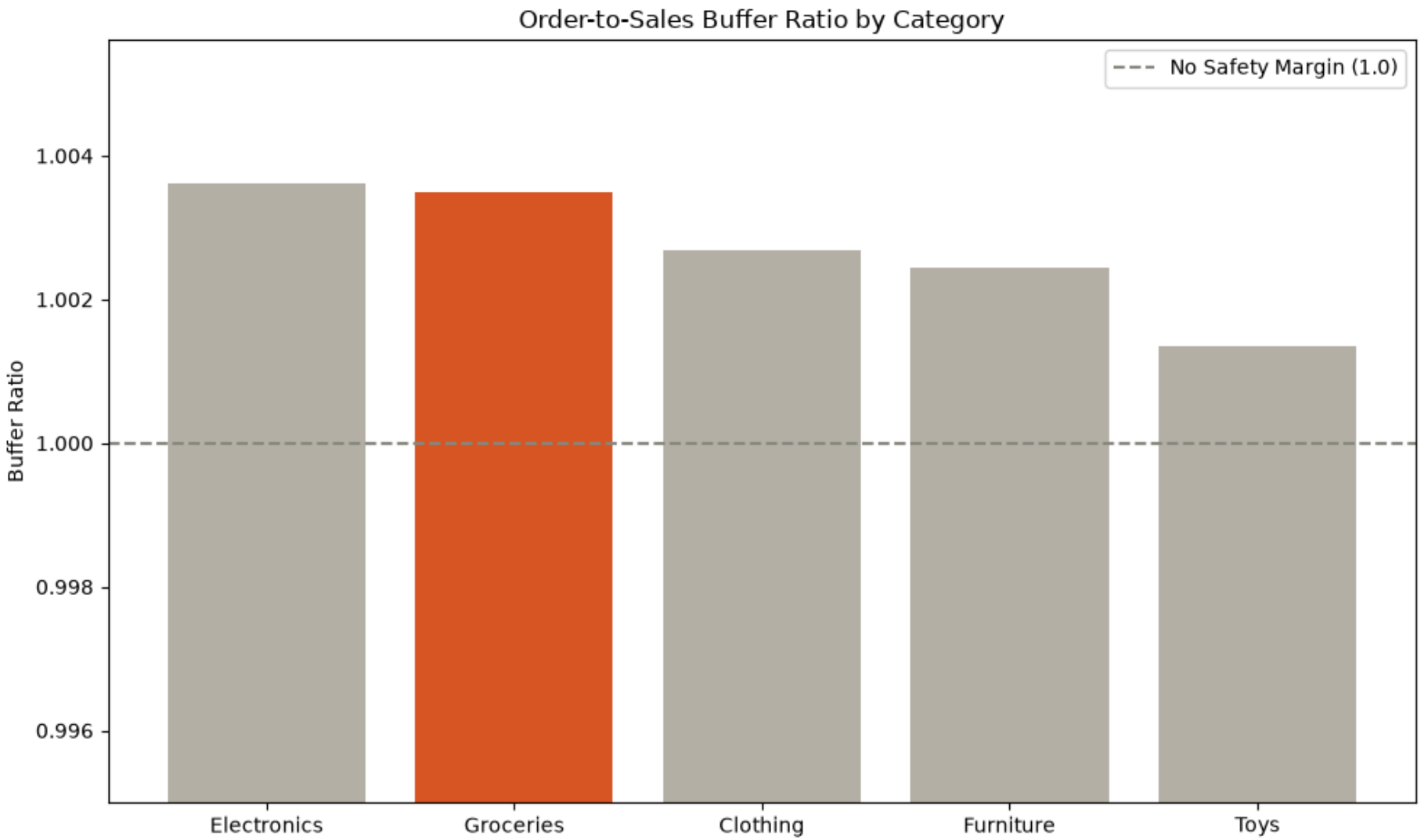
```
In [20]: import matplotlib.pyplot as plt

category_summary_sorted = category_summary.sort_values('Buffer_Ratio', ascending=False)

cats = list(category_summary_sorted.index.astype(str))
values = list(category_summary_sorted['Buffer_Ratio'])

colors = ['#D95926' if cat == 'Groceries' else '#B4B2A9' for cat in cats]

plt.figure(figsize=(10,6))
plt.bar(cats, values, color=colors)
plt.axhline(y=1.0, color='#898781', linestyle='--', linewidth=1.5, label='No Safety Margin (1.0)')
plt.ylim(0.995, max(values) + 0.002)
plt.title('Order-to-Sales Buffer Ratio by Category')
plt.ylabel('Buffer Ratio')
plt.legend()
plt.tight_layout()
plt.show()
```



✦ Insight

The order-to-sales ratio sits almost exactly at 1.0 across all five categories (ranging between 1.0014 and 1.0036), meaning order quantities nearly match historical average sales with virtually no real safety margin. This is not a Groceries-specific issue — it is a company-wide operating policy. Because Groceries has the highest sales volume and highest day-to-day volatility among all categories, the same blanket policy hits it the hardest, which explains why shortage incidents concentrate there.

Key Finding

Across all five categories, the order-to-sales ratio sits almost exactly at **1.0** (ranging between 1.0014 and 1.0036). This means order quantities are essentially set equal to the historical average sales, with virtually no real safety margin built in.

This is **not a Groceries-specific issue** — it is a company-wide ordering policy. However, because Groceries carries the highest sales volume and the highest day-to-day volatility among all categories, the same blanket policy hits it the hardest, which explains why stock shortage incidents concentrate there.

Recommendation

Instead of setting order quantities equal to the average historical sales, the ordering policy should include a **safety buffer** that reflects how much each category's demand actually fluctuates. Categories with higher day-to-day variability (like Groceries) need a proportionally larger buffer than more stable categories (like Furniture).

Next steps:

- Design a data-driven formula to calculate the appropriate safety buffer per category
- Prioritize Groceries first, given its scale and volatility
- Extend the same logic down to the store and product level once validated at the category level

Conclusion

Bottom line: The company does not need a fundamental change in sales or marketing strategy — it needs a targeted adjustment to how order quantities are calculated, so that each category carries a safety margin grounded in its actual demand variability, rather than relying on the historical average alone.